МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт Компьютерных наук и Кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Защита информации

Архитектура программного обеспечения

«Разработка межсетевого экрана»

Студенты группы
№5130201/30102: _____ Смирнов С.Р.

Михайлова А.А.

Аббад М.С.

Преподаватель: _____ Силиненко А.В.

«____»_____ 20___г.

Санкт-Петербург 2026

Содержание

1 Общая структура **3**

2 Подсистема фильтрации **6**

2.1 Тракт пакета 6

2.2 Почему DNAT стоит до политики, а SNAT после 7

2.3 Уже известный сеанс 7

2.4 Связи с другими подсистемами 8

3 Подсистема управления **8**

3.1 Точки входа 8

3.2 Единый API управления 9

3.3 Прикладные сервисы 9

4 Подсистема регистрации **10**

5 Подсистема контроля целостности **10**

6 Архитектура подсистемы контроля целостности **11**

7 Постоянные данные **14**

8 ОС Linux и жизненный цикл **15**

9 Реализация на Python 3.13 **15**

9.1 Процессы 15

9.2 Тракт фильтрации на Python 16

9.3 Используемые модули 17

1 Общая структура

ПО МЭ состоит из четырёх подсистем, которые выделены в ОБЩ-04: фильтрации, управления, регистрации и контроля целостности. Им служат два вспомогательных слоя: постоянные данные и среда ОС Linux. Схема приведена на рисунке 1, назначение частей кратко описано в таблице 1.

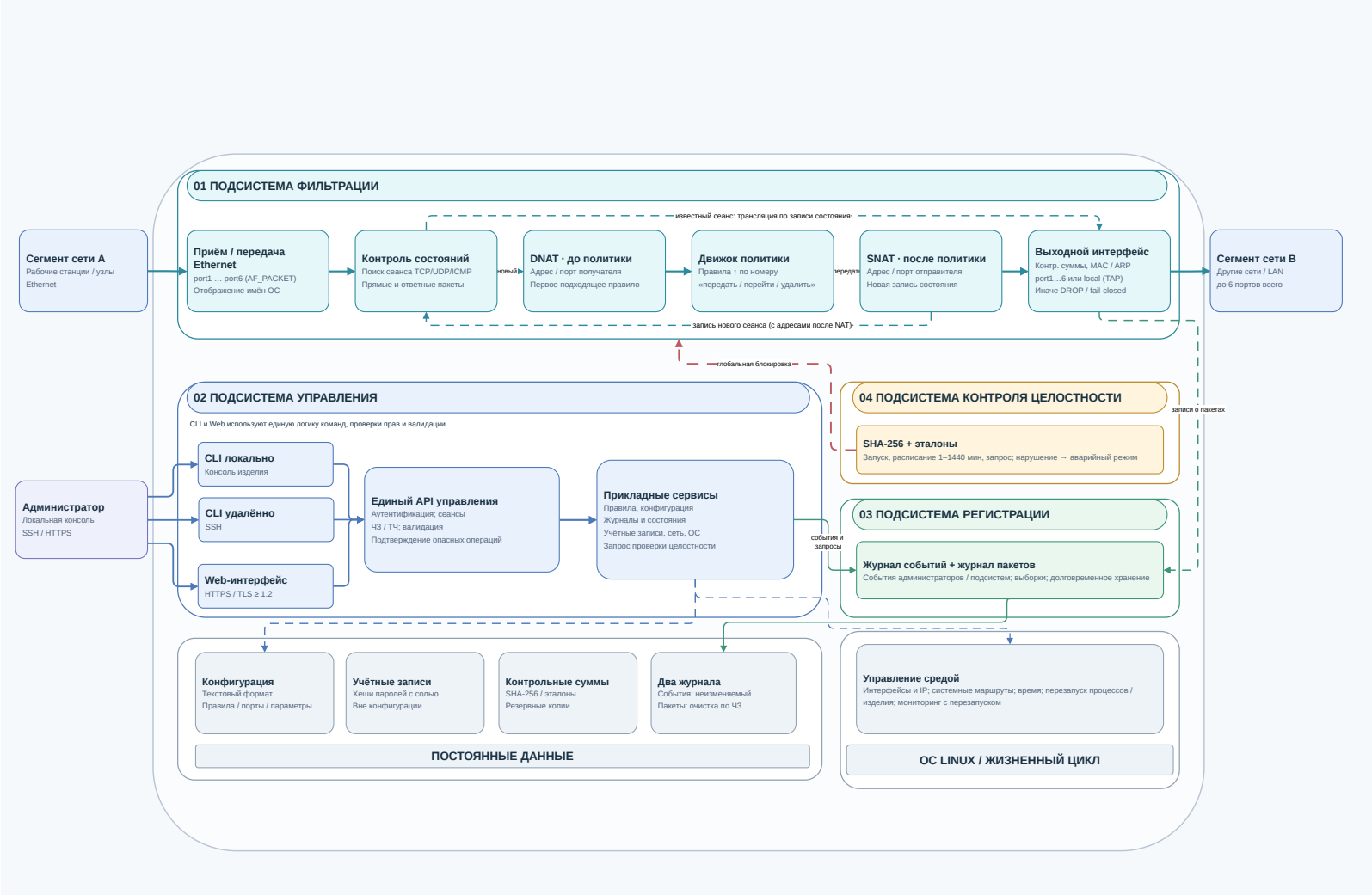


Рис. 1 – Архитектура ПО МЭ

Таблица 1 – Части ПО МЭ

Часть	**Назначение**
01 Подсистема фильтрации	Принимает кадры с портов, ищет сеанс в таблице состояний, выполняет DNAT, проверяет политику доступа, выполняет SNAT и передаёт кадр в выходной порт. Работает непрерывно и не зависит от того, работает ли управление.
02 Подсистема управления	Три точки входа администратора (CLI на консоли, CLI по SSH, web-интерфейс по HTTPS) и общее ядро: единый API управления и прикладные сервисы. Только через неё администратор меняет состояние ПО МЭ.
03 Подсистема регистрации	Принимает события от подсистем и записи о пакетах от фильтрации, хранит журналы, выполняет выборки.
04 Подсистема контроля целостности	Считает SHA-256 контролируемых файлов, сравнивает с эталонами и при нарушении включает глобальную блокировку трафика.
Постоянные данные	Конфигурация, учётные записи, эталонные контрольные суммы с резервными копиями, два журнала. Всё хранится на диске и переживает перезагрузку.
ОС Linux / жизненный цикл	Интерфейсы, IP-адреса, маршруты, время, запуск и перезапуск процессов, выключение и перезагрузка изделия.

Архитектура построена на трёх принципах.

1) **Каждая подсистема — отдельный процесс.** Подсистему можно остановить и перезапустить, не трогая остальные (ОБЩ-04, НДЖ-01). Ошибка в web-интерфейсе не останавливает фильтрацию.

2) **Одна логика для CLI и web.** Точки входа только принимают команды и показывают результат. Проверка прав, валидация и выполнение сосредоточены в едином API управления, поэтому функции в CLI и web совпадают (УПР-02), а правила проверяются одинаково.

3) **Отказ в «закрытое» состояние.** Если фильтрация не работает или включена глобальная блокировка, трафик через изделие не проходит (НДЖ-02, ЦЛС-04).

2 Подсистема фильтрации

2.1 Тракт пакета

Кадр проходит через шесть блоков слева направо (верхняя часть рисунка 1). Блоки перечислены в таблице 2.

Таблица 2 – Блоки тракта фильтрации

№	Блок	Что делает	Требования
1	Приём / передача Ethernet	Получает кадр с порта `port1` … `port6` через сокет `AF_PACKET`, ставит метку времени, разбирает заголовки, собирает фрагменты. При включённой глобальной блокировке кадр сразу удаляется. Тот же блок отвечает за сопоставление логических имён портов именам интерфейсов ОС.	ОБР-01, ОБР-03, ОБР-06, ОБР-07, ЦЛС-04
2	Контроль состояний	Ищет сеанс в таблице состояний (TCP, UDP, ICMP) в прямом и обратном направлении. Если сеанс известен, кадр транслируется по записи и сразу уходит в выходной интерфейс. Если нет — продолжает путь как новый.	СОС-02, СОС-03
3	DNAT (до политики)	Проверяет таблицу DNAT; первое подходящее правило заменяет адрес и порт получателя.	ФЛТ-10, НАТ-01 – НАТ-04
4	Движок политики	Просматривает правила по возрастанию номеров и применяет первое подходящее активное правило: «передать», «перейти» или «удалить». Политика видит адрес получателя *после* DNAT.	ФЛТ-01 – ФЛТ-09
5	SNAT (после политики)	Для пропущенного кадра проверяет таблицу SNAT и заменяет адрес и порт отправителя. Если у правила политики включён контроль состояния, создаёт запись нового сеанса с исходными и транслированными адресами.	ФЛТ-10, ФЛТ-11, СОС-01

№	Блок	Что делает	Требования
6	Выходной интерфейс	Пересчитывает контрольные суммы, определяет MAC-адрес следующего узла (ARP, маршруты) и передаёт кадр в назначенные порты или в `local` (TAP-интерфейс к собственным службам изделия). Если порт выключен или не назначен, кадр удаляется.	ФЛТ-08, ОБР-05, ОБР-08

2.2 Почему DNAT стоит до политики, а SNAT после

Такой порядок задан в НАТ-02: сначала DNAT, затем политика с учётом изменённого адреса, затем SNAT. Смысл его в следующем.

DNAT до политики. Пусть извне приходит запрос на адрес изделия `203.0.113.5:8080`, а правило DNAT перенаправляет его на внутренний сервер `192.168.1.10:80`. Поскольку DNAT выполнен раньше, политика видит реальный адрес получателя `192.168.1.10:80`. Администратор пишет правило «разрешить TCP к `192.168.1.10` порт 80», то есть описывает доступ к реальному серверу, а не к адресу, который в сети нигде не существует. Кроме того, от адреса получателя после DNAT зависит выбор выходного порта.

SNAT после политики. SNAT заменяет адрес отправителя, например внутренние адреса `192.168.1.0/24` на внешний адрес изделия. Если выполнять его после политики, то политика видит исходного отправителя и может разрешать или запрещать доступ по внутренним подсетям и конкретным узлам. Кроме того, для SNAT с адресом выходного интерфейса нужно знать этот интерфейс, а его выбирает только политика (действие «передать» с указанием выходных портов).

В результате каждый вид трансляции выполняется ровно там, где для него есть вся нужная информация, а политика всегда работает с «настоящими» адресами узлов.

2.3 Уже известный сеанс

Верхняя пунктирная стрелка показывает быстрый путь. Для пакетов уже установленного сеанса DNAT, политика и SNAT повторно не выполняются: запись в таблице состояний уже хранит обе пары адресов, до и после трансляции.

- Пакеты в прямом направлении получают ту же трансляцию, что и первый пакет сеанса.

- Ответные пакеты получают обратную трансляцию: например, ответ сервера `192.168.1.10:80` возвращается клиенту с адресом отправителя `203.0.113.5:8080`.

Нижняя пунктирная стрелка (от SNAT к контролю состояний) показывает, когда запись создаётся: *после* SNAT. Только в этот момент известны и исходные, и окончательные адреса, и выходной порт.

Быстрый путь важен и для производительности: полный просмотр DNAT, политики и SNAT выполняется один раз на сеанс, а не на каждый пакет (раздел 9.2).

2.4 Связи с другими подсистемами

- **Регистрация.** Если у правила стоит флаг регистрации, после обработки кадра формируется запись о пакете и отправляется в подсистему регистрации (стрелка «записи о пакетах»). Отправка не ждёт ответа: при переполнении буфера запись теряется, а счётчик потерь растёт, но фильтрация не тормозит (ОБР-10).

- **Контроль целостности.** Стрелка «глобальная блокировка» включает режим, в котором блок 1 удаляет все кадры (ЦЛС-04).

- **Управление.** Прикладные сервисы меняют правила через API политики (АПИ-01 – АПИ-12). Новая версия таблиц собирается целиком и заменяет старую одним действием (СТР-07): пакет обрабатывается либо по старой, либо по новой версии, но никогда по смеси.

- **Собственные службы изделия.** SSH и HTTPS администратора приходят на физический порт, проходят политику и с выходом `local` попадают в TAP-интерфейс, а оттуда в стек ОС к sshd и web-серверу (ОБР-05). Значит, удалённое управление тоже защищено политикой.

3 Подсистема управления

3.1 Точки входа

- **CLI локально** — консоль изделия (монитор и клавиатура или последовательный порт). Нужна для первоначальной настройки, когда в политике ещё нет правил для удалённого доступа, и для аварийного режима (ЦЛС-05).

- **CLI удалённо** — та же программа CLI, запущенная через SSH (ДСТ-07). Шифрование канала обеспечивает SSH.

- **Web-интерфейс** — HTTPS, TLS 1.2 и выше (ДСТ-07): страницы и запросы к единому API.

Точки входа не хранят данных и не принимают решений. Каждое действие администратора они превращают в вызов единого API и показывают ответ.

3.2 Единый API управления

Через этот блок проходит каждая команда, откуда бы она ни пришла.

- **Аутентификация и сеансы.** Вход по идентификатору и паролю (ДСТ-01), хранение сеансов, тайм-аут неактивности (ДСТ-06), принудительный сброс (УПР-12).
- **Права ЧЗ/ТЧ.** Для каждой функции проверяется уровень прав (ДСТ-03). Сеансы CLI и web хранятся в одном месте, поэтому правило «второй администратор с ЧЗ получает ТЧ» выполняется независимо от того, через какой интерфейс он вошёл (ДСТ-05).
- **Валидация.** Правила и параметры проверяются до выполнения (ФЛТ-05, НАТ-12, НДЖ-04). Ошибки возвращаются одинаковым текстом в CLI и web.
- **Подтверждение опасных операций.** Выключение, перезагрузка, применение конфигурации, очистка журнала пакетов и удаление учётной записи выполняются только после подтверждения (УПР-15).

3.3 Прикладные сервисы

Прикладные сервисы выполняют проверенные команды и обращаются к нужной подсистеме или данным (стрелки из блока на рисунке 1):

- **правила** — в API политики подсистемы фильтрации;
- **конфигурация** — сохранение, вывод и применение (УПР-09 – УПР-11), чтение и запись в постоянные данные;
- **журналы и состояния** — выборки из журналов (УПР-04 – УПР-07) через подсистему регистрации, таблица состояний (УПР-08) через фильтрацию;
- **учётные записи** — управление администраторами (УПР-12);
- **сеть и ОС** — интерфейсы, адреса, маршруты, время, перезапуск подсистем и изделия (УПР-13, УПР-14) через блок «Управление средой»;

- **запрос проверки целостности** — запуск проверки по команде администратора (ЦЛС-03).

Кроме того, сервисы отправляют в подсистему регистрации события о действиях администраторов: входах, изменениях политики и параметров (РЕГ-01, РЕГ-02).

4 Подсистема регистрации

Подсистема принимает данные из двух источников:

- **события** — от подсистемы управления и остальных подсистем (запуск, остановка, аварийное завершение, контроль целостности) с кодами из приложения А ТЗ;
- **записи о пакетах** — от подсистемы фильтрации, отдельным потоком.

Она же выполняет выборки по запросам прикладных сервисов: по коду и времени для событий, по действию, интерфейсам, правилу, адресам и портам для пакетов (УПР-04, УПР-05). Журналы хранятся в постоянных данных. Журнал событий средствами управления не изменяется, журнал пакетов очищается только администратором с правами ЧЗ (РЕГ-05).

5 Подсистема контроля целостности

- **Что проверяется:** исполняемые файлы и конфигурационные файлы ПО МЭ, включая сохранённые конфигурации и базу учётных записей (ЦЛС-01).
- **Как:** SHA-256 каждого файла сравнивается с эталоном из постоянных данных.
- **Когда:** при запуске, по расписанию (от 1 до 1440 минут) и по запросу администратора (ЦЛС-03).
- **Легитимные изменения.** Когда управление само меняет конфигурационный файл (сохранение конфигурации, изменение политики, учётных записей), эталон и резервная копия обновляются (ЦЛС-02).
- **При нарушении:** событие с перечнем файлов, глобальная блокировка трафика и переход в аварийный режим (ЦЛС-04). В аварийном режиме доступен только локальный CLI с функциями восстановления из резервных копий (ЦЛС-05). После успешной повторной проверки блокировка снимается (ЦЛС-06).

6 Архитектура подсистемы контроля целостности

Подсистема реализована процессом `fw-integrity` на Python 3.13, используется только стандартная библиотека. Процесс работает от `root`: ему нужно читать все контролируемые файлы, восстанавливать их и останавливать службы в аварийном режиме. Схема приведена на рисунке 2, назначение компонентов — в таблице 3.

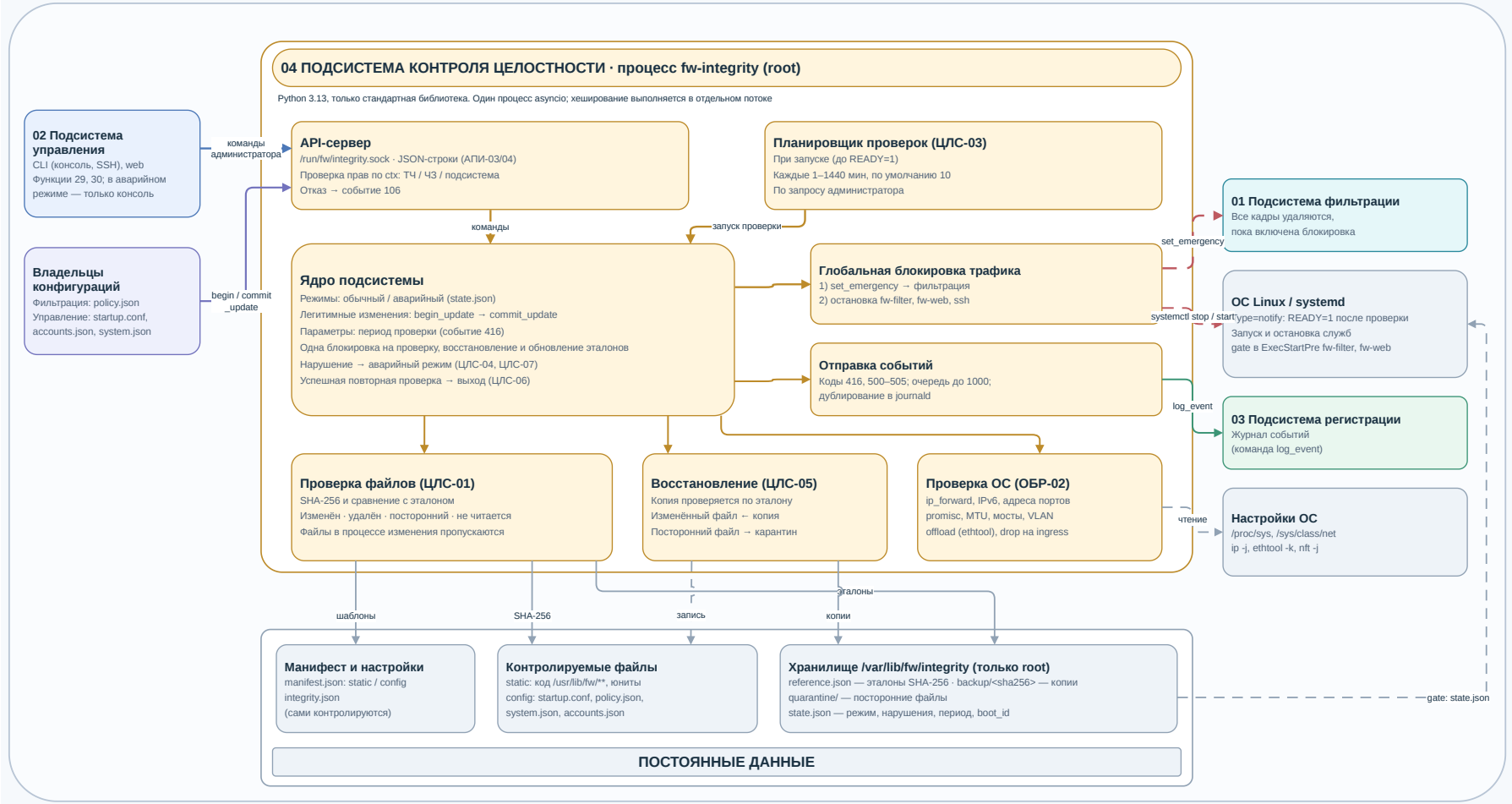


Рис. 2 – Архитектура подсистемы контроля целостности

Таблица 3 – Компоненты подсистемы контроля целостности

Компонент	**Модуль**	**Назначение**
API-сервер	`protocol`	UNIX-сокет `/run/fw/integrity.sock`, JSON-строки (АПИ-03, АПИ-04), коды ошибок (АПИ-09). Права проверяются по контексту вызова: ТЧ, ЧЗ или подсистема; отказ регистрируется событием 106
Планировщик	`daemon`	Проверка при запуске до сигнала готовности systemd, каждые 1 – 1440 мин (по умолчанию 10) и по запросу (ЦЛС-03)
Ядро	`service`	Команды API, режимы «обычный» и «аварийный», легитимные изменения файлов (ЦЛС-02), период проверки. Проверка, восстановление и обновление эталонов выполняются по очереди под одной блокировкой
Проверка файлов	`checker`, `manifest`	SHA-256 каждого файла с эталоном; поиск изменённых, удалённых, посторонних (в том числе скрытых) и нечитаемых файлов (ЦЛС-01)
Проверка ОС	`os_state`	Сверка с таблицей ОБР-02: пересылка ядром, IPv6, адреса портов, promisc и MTU, мосты и VLAN, аппаратные функции, правило `drop` на `netdev ingress`
Восстановление	`store`	Изменённый или удалённый файл восстанавливается из резервной копии после её проверки по эталону; посторонний файл перемещается в карантин (ЦЛС-05)
Глобальная блокировка	`actions`	Команда `set_emergency` подсистеме фильтрации и остановка служб fw-filter, fw-web, ssh (ЦЛС-04); при выходе — обратные действия (ЦЛС-06)
Отправка событий	`events`	Команда `log_event` подсистеме регистрации (коды 416, 500 – 505); очередь до 1000 событий, если регистрация недоступна

Режимы работы. В обычном режиме трафик обрабатывается, а файлы проверяются по расписанию. Любое нарушение (или невозможность проверки: нет эталонов,

повреждён манифест) переводит подсистему в аварийный режим. Сначала режим записывается в `state.json`, затем включается глобальная блокировка, затем регистрируются события 501 и 502. Возврат в обычный режим происходит только после повторной проверки без нарушений (событие 504). Запуск fw-filter и fw-web защищён проверкой `fw-integrity gate` в `ExecStartPre`: служба стартует, только если в текущей загрузке уже была успешная проверка (ЦЛС-07).

Легитимные изменения. Подсистемы, которые пишут конфигурационные файлы (политику, сохранённую конфигурацию, учётные записи), вызывают `begin_update`, атомарно записывают файл и вызывают `commit_update`. Пока изменение не завершено, файл не проверяется. После `commit_update` эталон и резервная копия обновляются (событие 505). Изменение без этих вызовов считается нарушением.

Данные подсистемы хранятся в каталоге `/var/lib/fw/integrity`, доступном только `root`:

- `reference.json` — эталонные SHA-256, вид, размер и права файлов;
- `backup/` — резервные копии, имя файла — его SHA-256;
- `quarantine/` — посторонние файлы, убранные при восстановлении;
- `state.json` — режим, перечень нарушений, период и идентификатор загрузки.

Перечень контролируемых файлов задаёт манифест `manifest.json`: `static` для кода и неизменяемых файлов, `config` для изменяемых конфигураций. Сам манифест и файл настроек всегда контролируются.

7 Постоянные данные

Таблица 4 – Постоянные данные

Хранилище	**Содержимое**	**Кто пишет**
Конфигурация	Текстовый файл: правила фильтрации, DNAT, SNAT, порты, параметры подсистем (УПР-10). Текущая и сохранённая, а также конфигурация по умолчанию	Прикладные сервисы

Хранилище	**Содержимое**	**Кто пишет**
Учётные записи	Идентификаторы, уровни прав, хеши паролей с солью (ДСТ-02). В конфигурацию не входят	Прикладные сервисы
Контрольные суммы	Эталонные SHA-256 и резервные копии контролируемых файлов	Подсистема контроля целостности
Два журнала	Журнал событий (неизменяемый) и журнал пакетов (очистка по ЧЗ)	Подсистема регистрации

8 ОС Linux и жизненный цикл

Блок «Управление средой» выполняет всё, что требует прав в ОС:

- приводит ОС в состояние из ОБР-02: у портов нет IP-адресов, они в режиме `promisc`, а правило nftables `netdev ingress drop` не пускает кадры портов в стек ядра;
- создаёт TAP-интерфейсы и назначает на них IP-адреса изделия (ОБР-05, УПР-13);
- управляет системными маршрутами и временем (УПР-14);
- запускает подсистемы, следит за ними и перезапускает после аварийного завершения с регистрацией события (НДЖ-01); перезапускает подсистему или изделие по команде администратора.

9 Реализация на Python 3.13

9.1 Процессы

Каждая подсистема реализуется отдельной программой на Python 3.13 и запускается как служба systemd. Подсистемы обмениваются сообщениями через UNIX-сокеты в каталоге `/run/fw/` в формате АПИ-03/04: одна JSON-строка на запрос или ответ.

Таблица 5 – Процессы и их соответствие схеме

Блок схемы	**Процесс**	**Реализация**	**Права в ОС**
01 Фильтрация	`fw-filter`	Один поток, цикл `selectors` по сокетам портов, TAP и сокету API политики	пользователь `fwfilter`, `CAP_NET_RAW`, `CAP_NET_ADMIN` (ОБР-12)
02 Единый API и прикладные сервисы	`fw-mgmt`	`asyncio`, сервер на `/run/fw/mgmt.sock`; сеансы, права, валидация, вызовы остальных подсистем	`root` (выполняет действия «Управления средой»)
02 CLI локально и удалённо	`fw-cli`	Клиент `mgmt.sock`; на консоли — оболочка входа, по SSH — `ForceCommand` в `sshd_config`	`fwcli`
02 Web-интерфейс	`fw-web`	`http.server.ThreadingHTTPServer` + `ssl`, страницы и JSON-запросы к `mgmt.sock`	`fwweb`, `CAP_NET_BIND_SERVICE`
03 Регистрация	`fw-log`	`asyncio`; `sqlite3` (режим WAL), базы событий и пакетов	`fwlog`
04 Контроль целостности	`fw-integrity`	`hashlib.sha256`, таймер проверок, резервные копии	`root` (чтение всех файлов)
ОС / жизненный цикл	systemd + `fw-mgmt`	Юниты с `Restart=on-failure`; команды `ip`, `nft`, `sysctl`, `ethtool`, `timedatectl` через `subprocess`	`root`

Перезапуск подсистем выполняет systemd, а события о нём фиксируются так: в юнит каждой подсистемы добавляются `ExecStartPost` и `ExecStopPost`, которые вызывают маленькую утилиту `fw-notify`. Она передаёт в `fw-log` событие запуска (210) или остановки (211, а при `$SERVICE_RESULT` $\neq$ `success` — 213) с кодом завершения из переменных `$EXIT_CODE` и `$EXIT_STATUS`.

9.2 Тракт фильтрации на Python

- **Один поток вместо многих.** Из-за GIL потоки Python не дают параллельной обработки, а таблица состояний должна быть общей для всех портов (ответ приходит на другой порт). Поэтому `fw-filter` обрабатывает все порты в од-

ном цикле `selectors` (epoll). Блокировки не нужны, и требование безопасного изменения таблиц (ОБР-12) выполняется автоматически.

- **Атомарная замена политики** (СТР-07). Команды API обрабатываются в том же цикле между кадрами. Новая версия таблиц собирается в отдельный объект, а затем одним присваиванием подменяет ссылку на текущую версию.

- **Сокеты портов.** `socket.socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))` с привязкой к интерфейсу. Обязательно устанавливается опция `PACKET_IGNORE_OUTGOING` (значение 23, уровень `SOL_PACKET` = 263). Иначе сокет получает обратно кадры, которые сам же отправил, и они пойдут по кругу.

- **TAP.** Открывается `/dev/net/tun` и вызывается `fcntl.ioctl` с `TUNSETIFF` (флаги `IFF_TAP | IFF_NO_PI`). Дескриптор добавляется в тот же цикл `selectors`.

- **Разбор без копирования.** Кадр читается в заранее выделенный буфер (`recv_into`), заголовки разбираются через `memoryview` и заранее скомпилированные `struct.Struct`.

- **Правила в «скомпилированном» виде.** При загрузке политики адреса переводятся в целые числа, подсети — в пары (адрес, маска), порты — в диапазоны. Проверка правила сводится к сравнению целых чисел.

- **Таблица состояний** — словарь с ключом (протокол, адреса, порты), по одной записи на каждое направление.

- **Контрольные суммы после NAT** пересчитываются инкрементально (RFC 1624): меняются только изменённые 16-битные слова, а не весь пакет.

Оценка производительности (ОБР-13). 100 Мбит/с при кадрах 1518 байт — это примерно 8 100 кадров в секунду (плюс около 4 000 коротких подтверждений TCP в обратную сторону). На один кадр приходится около 80 мкс. Пакеты установленного сеанса идут по быстрому пути (поиск в словаре, трансляция, отправка), и это укладывается в бюджет даже на одном ядре. Полный просмотр 100 правил DNAT, политики и SNAT выполняется только для первого пакета сеанса. Поэтому для теста `iperf3` правило должно быть с контролем состояния. Оценку нужно подтвердить замером на стенде.

9.3 Используемые модули

Используется только стандартная библиотека Python, поэтому дополнительные пакеты устанавливать не нужно.

Таблица 6 – Модули стандартной библиотеки

Модуль	**Для чего**
`socket`, `selectors`	`AF_PACKET`, UNIX-сокеты, цикл обработки кадров
`fcntl`, `os`	создание TAP-интерфейсов, чтение и запись дескриптора
`struct`, `ipaddress`	разбор заголовков, адреса и подсети
`asyncio`	серверы `fw-mgmt` и `fw-log`
`json`	протокол обмена и формат конфигурации
`hashlib`	SHA-256 (ЦЛС-01); `hashlib.scrypt` для хешей паролей с солью (ДСТ-02)
`secrets`, `hmac`	токены сеансов, сравнение без утечки по времени
`ssl`, `http.server`	HTTPS-сервер, `minimum_version = TLSv1_2` (ДСТ-07)
`sqlite3`	журналы событий и пакетов, выборки по критериям
`subprocess`	вызов `ip`, `nft`, `sysctl`, `ethtool`, `timedatectl`, `systemctl`
`logging`, `argparse`	отладочный вывод и разбор аргументов CLI